Рассмотрение методов предотвращения подгрузки DLL

Возможно ли избавиться от чужих модулей? Как предотвратить загрузку библиотек или хотя бы своевременно о ней узнать? Давай разбираться.

В основном Windows использует два формата исполняемых файлов — .exe и DLL.

Первые представляют собой привычные для нас программы, а DLL — это библиотеки с дополнительными функциями, подгружаемые исполняемым файлом.

Файлы DLL могут быть загружены в процесс и сразу при запуске (так называемая статическая линковка), и при выполнении (динамическая линковка),

то есть когда процесс уже вовсю работает и вдруг что-то его подталкивает к решению загрузить библиотеку

Процессы в Windows имеют виртуальное адресное пространство.

Пространство представляет собой участок памяти, который принадлежит только текущему процессу.

В этой части памяти будут данные только текущего процесса.

При подгрузке DLL оказывается в том же самом адресном пространстве и, как следствие, получает доступ ко всем данным процесса.

Такое поведение приводит к тому, что становится возможным подгрузки вредоносной библиотеки в процесс.

Рассмотрим наиболее распространенных способов предотвращения подгрузки DLL-библиотек в процесс.

UpdateProcThreadAttribute

Первый вариант основан на функции UpdateProcThreadAttribute().

Для его реализации устанавливается специальное значение: PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY

Единственная проблема – в нашу программу все-таки смогут внедряться чужие библиотеки, но только если они подписаны в Microsoft.

Также стоит отметить, что предотвращение подгрузки DLL будет применяться к процессу, который будет запущен через **CreateProcess()**, к текущему процессу применить эту функцию не получится.

Тем не менее есть возможность обойти это ограничение. Наш текущий процесс может проверять, был ли он запущен с параметром, ограничивающим подгрузку DLL.
Если нет, то он запускает копию самого себя, но уже с помощью атрибутов, которые устанавливаются в функции

UpdateProcThreadAttribute().

Реализуем функцию CreateProcessWithBlockDllPolicy(). Для этого нам потребуется лишь путь до исполняемого файла, который нужно запустить. Функция будет возвращать PID запущенного процесса, его хендл, а также хендл потока процесса.

```
BOOL CreateProcessWithBlockDllPolicy(IN LPSTR lpProcessPath, OUT DWORD* dwProcessId,
OUT HANDLE* hProcess, OUT HANDLE* hThread) {
	STARTUPINFOEXA    SiEx = { 0 };
	PROCESS_INFORMATION   Pi = { 0 };
	SIZE_T        sAttrSize = NULL;
	if (lpProcessPath == NULL)
		return FALSE;
	RtlSecureZeroMemory(&SiEx, sizeof(STARTUPINFOEXA));
	RtlSecureZeroMemory(&Pi, sizeof(PROCESS_INFORMATION));
	SiEx.StartupInfo.cb = sizeof(STARTUPINFOEXA);
	SiEx.StartupInfo.dwFlags = EXTENDED_STARTUPINFO_PRESENT;
	InitializeProcThreadAttributeList(NULL, 1, NULL, &sAttrSize);
	LPPROC_THREAD_ATTRIBUTE_LIST pAttrBuf =
(LPPROC_THREAD_ATTRIBUTE_LIST)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, sAttrSize);
	if (!InitializeProcThreadAttributeList(pAttrBuf, 1, NULL, &sAttrSize)) {
		printf("[!] InitializeProcThreadAttributeList Failed With Error : %d \n",
GetLastError());
		return FALSE;
	}
	DWORD64 dwPolicy =
PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON;
	if (!UpdateProcThreadAttribute(pAttrBuf, NULL,
PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY, &dwPolicy, sizeof(DWORD64), NULL, NULL)) {
		printf("[!] UpdateProcThreadAttribute Failed With Error : %d \n",
GetLastError());
		return FALSE;
	}
	SiEx.lpAttributeList = (LPPROC_THREAD_ATTRIBUTE_LIST)pAttrBuf;
	if (!CreateProcessA(
			NULL,
			lpProcessPath,
			NULL,
			NULL,
			FALSE,
			EXTENDED_STARTUPINFO_PRESENT,
			NULL,
			NULL,
			&SiEx.StartupInfo,
			&Pi)) {
		printf("[!] CreateProcessA Failed With Error : %d \n", GetLastError());
		return FALSE;
	}
	*dwProcessId = Pi.dwProcessId;
	*hProcess = Pi.hProcess;
	*hThread = Pi.hThread;
	DeleteProcThreadAttributeList(pAttrBuf);
	HeapFree(GetProcessHeap(), 0, pAttrBuf);
	if (*dwProcessId != NULL && *hProcess != NULL && *hThread != NULL)
		return TRUE;
	else
		return FALSE;
}
```

Сначала мы очищаем структуры, а затем инициализируем структуры, необходимые для запуска процесса. Дополнительно устанавливаем флаг: EXTENDED_STARTUPINFO_PRESENT

Это даст возможность запускать процессы с нужной митигацией.

Далее с помощью функции InitializeProcThreadAttributeList() готовим структуру LPPROC_THREAD_ATTRIBUTE_LIST.

Ее будем указывать в функцию UpdateProcAttributeList(). Двойной вызов функции стандартен для Windows – первый вызов служит для получения нужного размера, а второй для инициализации элементов структуры.

Затем с помощью функции UpdateProcAttributeList() заносим необходимые атрибуты в нашу структуру со списком атрибутов, после чего применяем ее в функции CreateProcessA(), что позволяет запустить процесс с защитой от подгрузки левых библиотек.

Как же запустить текущий процесс, применив эту функцию?

Здесь будем использовать аргументы **командной строки**.

Наш процесс может породить дочерний такой же процесс с митигацией, а самого себя уничтожить.

Для этого нам нужно создать специальный STOP_ARG.

Если вдруг в аргументах командной строки текущего процесса нет STOP_ARG, то он должен перезапуститься с защитой от подгрузки DLL.

Функция main() должна выглядеть вот так:

```
#define LOCAL_BLOCKDLLPOLICY
#ifdef LOCAL_BLOCKDLLPOLICY
#define STOP_ARG "xakep"
#endif
int main(int argc, char* argv[]) {
	DWORD   dwProcessId = NULL;
	HANDLE  hProcess = NULL,
		hThread = NULL;
#ifdef LOCAL_BLOCKDLLPOLICY
	if (argc == 2 && (strcmp(argv[1], STOP_ARG) == 0)) {
		printf("[+] Process Is Now Protected With The Block Dll Policy
\n");
		WaitForSingleObject((HANDLE)-1, INFINITE);
	}
	else {
		printf("[!] Local Process Is Not Protected With The Block Dll
Policy \n");
		CHAR pcFilename[MAX_PATH * 2];
		if (!GetModuleFileNameA(NULL, (LPSTR)&pcFilename, MAX_PATH * 2)) {
			printf("[!] GetModuleFileNameA Failed With Error : %d \n",
GetLastError());
			return -1;
		}
		DWORD dwBufferSize = (DWORD)(lstrlenA(pcFilename) +
lstrlenA(STOP_ARG) + 0xFF);
		CHAR* pcBuffer = (CHAR*)HeapAlloc(GetProcessHeap(),
HEAP_ZERO_MEMORY, dwBufferSize);
		if (!pcBuffer)
			return FALSE;
		sprintf_s(pcBuffer, dwBufferSize, "%s %s", pcFilename, STOP_ARG);
		if (!CreateProcessWithBlockDllPolicy(pcBuffer, &dwProcessId,
&hProcess, &hThread)) {
			return -1;
		}
		HeapFree(GetProcessHeap(), 0, pcBuffer);
		printf("[i] Process Created With Pid %d \n", dwProcessId);
	}
#endif
#ifndef LOCAL_BLOCKDLLPOLICY
	if
(!CreateProcessWithBlockDllPolicy((LPSTR)"C:\\Windows\\System32\\RuntimeBroker.
exe", &dwProcessId, &hProcess, &hThread)) {
		return -1;
	}
	printf("[i] Process Created With Pid %d \n", dwProcessId);
#endif
	return 0;
}
```

Сначала объявляются некоторые директивы, позволяющие управлять тем, что мы хотим получить от кода.

Если хотим защитить собственную программу, то оставляем define LOCAL_BLOCKDLLPOLICY.

Если хотим запустить иной процесс с защитой от подгрузки DLL, то убираем это поле.

Затем указываем необходимый STOP_ARG, а в самой функции main() проверяем, присутствует ли STOP_ARG в аргументах командной строки.

Если он есть, то процесс может спокойно продолжать свое выполнение.

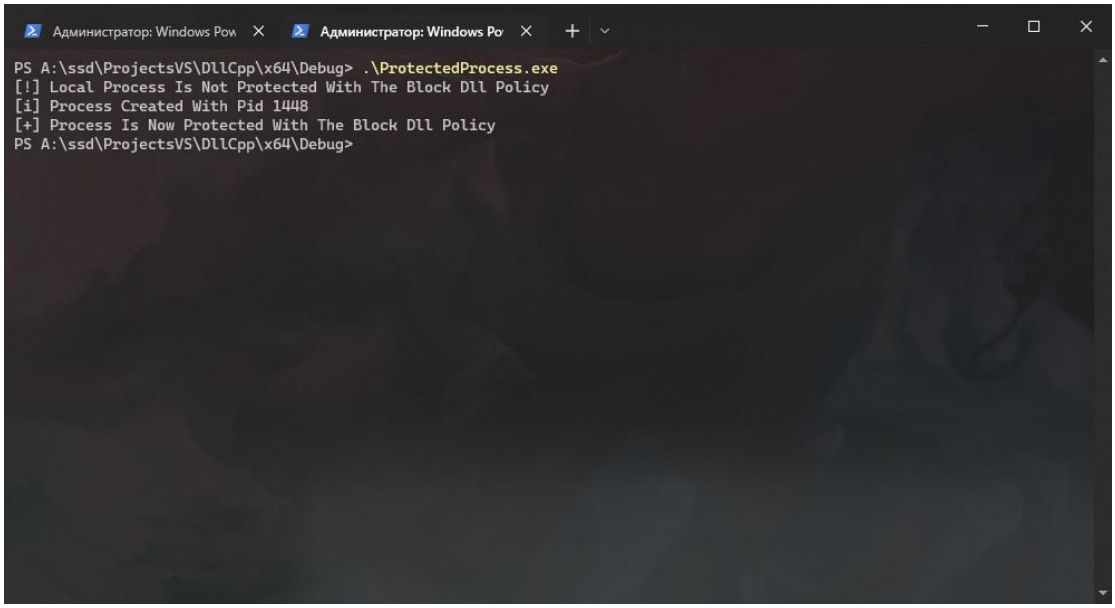
```
if (argc == 2 && (strcmp(argv[1], STOP_ARG) == 0)) {
printf("[+] Process Is Now Protected With The Block Dll Policy \n");
// Продолжаем выполнение
// Наш код тут
WaitForSingleObject((HANDLE)-1, INFINITE);
}
```

Если же этого аргумента нет, то процесс порождает сам себя с защитой.

```
printf("[!] Local Process Is Not Protected With The Block Dll Policy \n");
CHAR pcFilename[MAX_PATH * 2];
if (!GetModuleFileNameA(NULL, (LPSTR)&pcFilename, MAX_PATH * 2)) {
printf("[!] GetModuleFileNameA Failed With Error : %d \n", GetLastError());
return -1;
}
DWORD dwBufferSize = (DWORD)(lstrlenA(pcFilename) + lstrlenA(STOP_ARG) + 0xFF);
CHAR* pcBuffer = (CHAR*)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, dwBufferSize);
if (!pcBuffer)
return FALSE;
sprintf_s(pcBuffer, dwBufferSize, "%s %s", pcFilename, STOP_ARG);
if (!CreateProcessWithBlockDllPolicy(pcBuffer, &dwProcessId, &hProcess, &hThread)) {
return -1;
}
HeapFree(GetProcessHeap(), 0, pcBuffer);
printf("[i] Process Created With Pid %d \n", dwProcessId);
```

Проверка

Запускаем наш процесс с защитой. Скомпилированный файл назван ProtectedProcess.exe.



Видно, что наш файл имеет особый бит, который показывает даже Process Hacker.(Утилита Process Haker https://systeminformer.sourceforge.io/downloads)

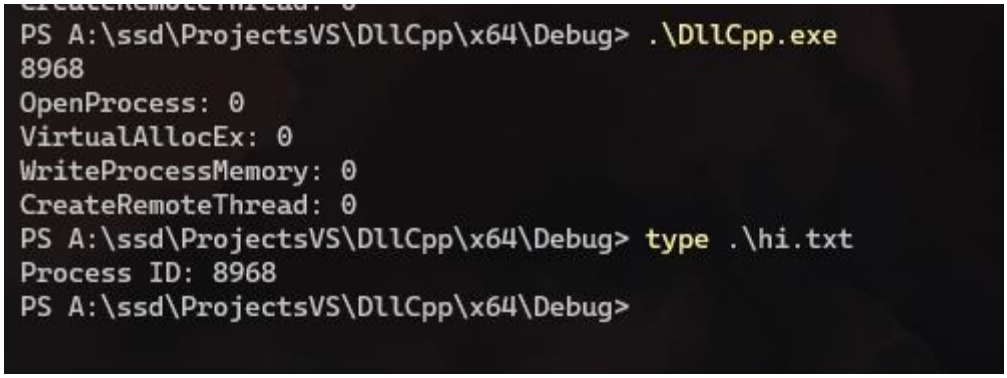
Теперь попробуем внедриться. Для этого можно использовать простейший инжектор на CreateRemoteThread().

```
#include <Windows.h>
#include <iostream>
int main(int argc, char* argv[]) {
	HANDLE processHandle;
	PVOID remoteBuffer;
	wchar_t dllPath[] = TEXT("A:\\SSD\\ProjectsVS\\DllCpp\\x64\\Debug\\DllInj.dll");
	DWORD pid;
	std::cin >> pid;
	processHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
	std::cout << "OpenProcess: " << GetLastError() << std::endl;
	remoteBuffer = VirtualAllocEx(processHandle, NULL, sizeof(dllPath),
MEM_RESERVE|MEM_COMMIT, PAGE_EXECUTE_READWRITE);
	std::cout << "VirtualAllocEx: " << GetLastError() << std::endl;
	WriteProcessMemory(processHandle, remoteBuffer, (LPVOID)dllPath,
sizeof(dllPath), NULL);
	std::cout << "WriteProcessMemory: " << GetLastError() << std::endl;
	PTHREAD_START_ROUTINE threatStartRoutineAddress =
(PTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandle(TEXT("Kernel32")),
"LoadLibraryW");
	CreateRemoteThread(processHandle, NULL, 0, threatStartRoutineAddress,
remoteBuffer, 0, NULL);
	std::cout << "CreateRemoteThread: " << GetLastError() << std::endl;
	CloseHandle(processHandle);
	return 0;
}
```

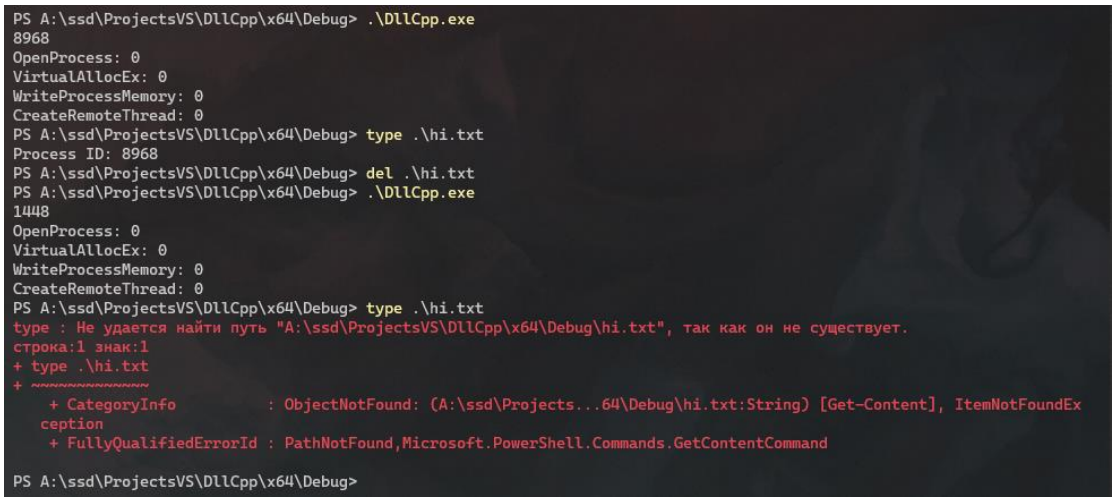
Инжектор загружает библиотеку со следующим кодом:

```
#include "pch.h"
#include <iostream>
#include <string>
BOOL WINAPI DllMain(HINSTANCE hinstDll, DWORD fdwReason, PVOID fImpLoad) {
switch (fdwReason) {
case DLL_PROCESS_ATTACH:
DWORD dwProcessId = GetCurrentProcessId();
HANDLE hFile = CreateFileA("A:\\ssd\\ProjectsVS\\DllCpp\\x64\\Debug\\hi.txt",
GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile != INVALID_HANDLE_VALUE)
{
std::string content = "Process ID: " + std::to_string(dwProcessId) + "\r\n";
DWORD bytesWritten;
WriteFile(hFile, content.c_str(), content.size(), &bytesWritten, NULL);
CloseHandle(hFile);
}
}
return(TRUE);
}
```

Сначала проверим работу на безобидном процессе, например GoogleCrashHandler.exe с PID 8968.



Теперь попробуем внедриться в защищенный процесс.



Неудача! Инжект не сработал.
Если попробуем внедриться через Process Hacker, тоже столкнемся с невозможностью выполнения нашей библиотеки.

SetProcessMitigationPolicy

Эту фун-кцию так-же мож-но исполь-зовать для пре-дот-вра-щения под-груз-ки в наш про-цесс левых DLL. Заметь, что подоб-ное пре-дот-вра-щение под-груз-ки либ при-меня-ется к текуще-му про-цес-су и толь-ко пос-ле вызова этой фун-кции. Если же хакер внед-ряет свою биб-лиоте-ку до вызова этой фун-кции, то у него все получит-ся. Если пос-ле, то ничего не вый-дет. И опять же единс-твен-ное исклю-чение — DLL, под-писан-ные в Microsoft. Они все рав-но без проб-лем смо-гут внед-рять-ся в наш про-цесс.

```
#include <Windows.h>
#include <iostream>
int main() {
	PROCESS_MITIGATION_BINARY_SIGNATURE_POLICY Struct = { 0 };
	Struct.MicrosoftSignedOnly = 1;
	if (!SetProcessMitigationPolicy(ProcessSignaturePolicy, &Struct,
sizeof(Struct))) {
		printf("[!] SetProcessMitigationPolicy Failed With Error : %d \n",
GetLastError());
	}
	getchar();
	return 0;
}
```